

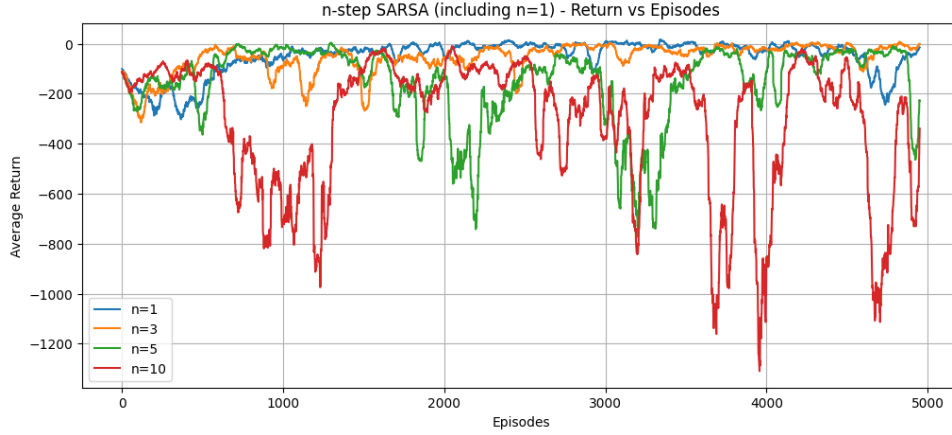
Multi-Agent Reinforcement Learning

Assignment 2

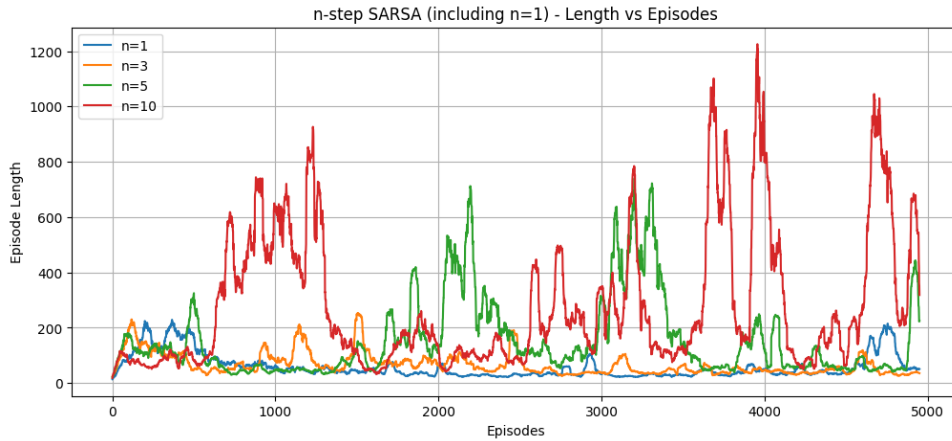
Sambit Sahoo (22277)

Please find the source code at [GitHub](#) or [Drive Link](#)

Q1- Implementing Questions of Assignment 1 in SARSA, nstep SARSA, comparing performances of n step sarsa as n increased.



(a) Average return vs episodes of sarsa , nstep sarsa



(b) Episode length vs episodes of sarsa , nstep sarsa

Figure 1: Comparison of SARSA and n-step SARSA as n increases

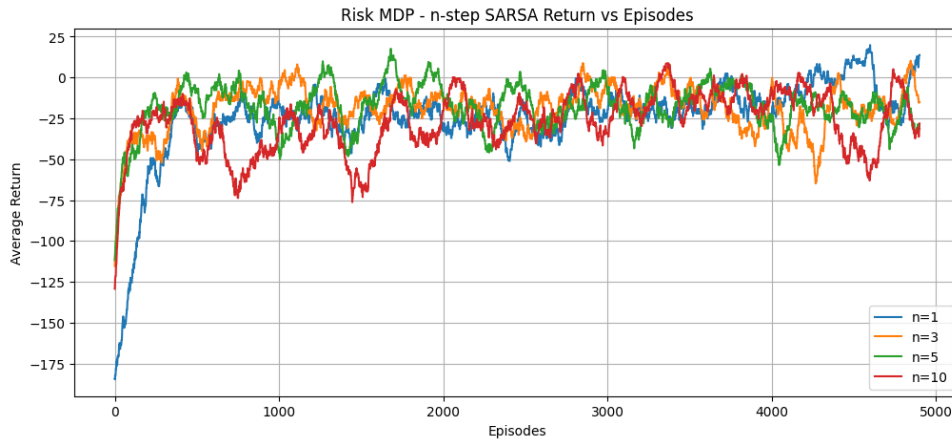
Method	Time (s)	Memory (KB)
n-step SARSA ($n=1$)	9.0024	200.71
n-step SARSA ($n=3$)	14.4089	251.03
n-step SARSA ($n=5$)	29.5984	832.87
n-step SARSA ($n=10$)	55.5909	1045.01

Table 1: Comparison of Time and Memory Usage across Methods

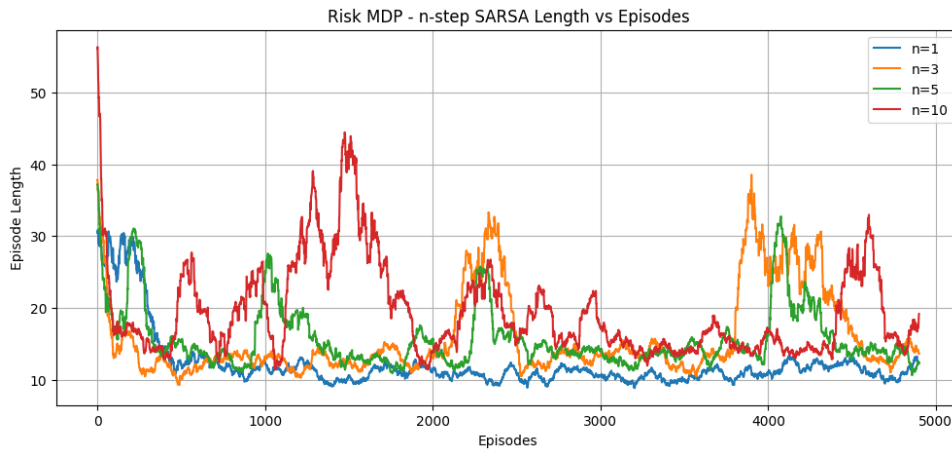
- Figure 1 shows the variation of average return and episode length for SARSA ($n = 1$) and n-step SARSA with different values of n .

- SARSA ($n = 1$) exhibits stable but relatively slower learning, with smooth improvement in return and consistently low episode lengths.
- For moderate values such as $n = 3$, the agent learns faster while maintaining stability, achieving a good balance between convergence speed and variance.
- As n increases further ($n = 5, n = 10$), learning becomes increasingly unstable.
- This instability is reflected in large fluctuations in both return and episode length, with frequent spikes indicating inefficient exploration and high variance updates.
- These observations illustrate the bias–variance trade-off in n -step methods:
 - Smaller n : more stable but higher bias
 - Larger n : lower bias but significantly higher variance

Similarly for question 3 of assignment 1:



(a) Average return vs episodes of sarsa , nstep sarsa



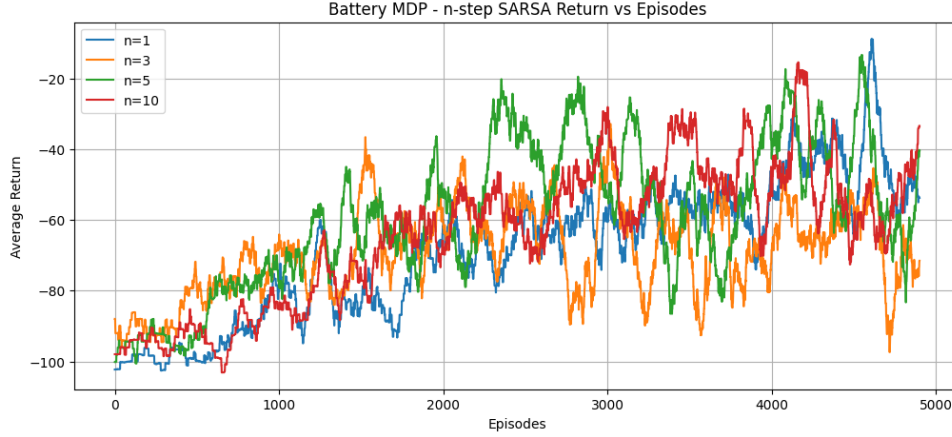
(b) Episode length vs episodes of sarsa , nstep sarsa

Figure 2: Comparison of SARSA and n -step SARSA as n increases

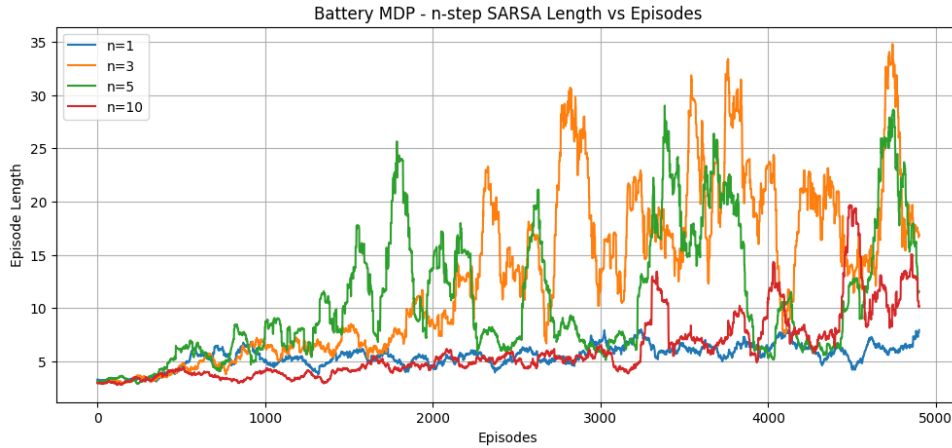
Method	Time (s)	Memory (KB)
SARSA	4.1620	140.73
n-step SARSA (n=3)	5.3741	126.50
n-step SARSA (n=5)	5.7942	125.23
n-step SARSA (n=10)	7.0238	133.23

Table 2: Comparison of Time and Memory Usage across Methods

Similarly for question 2 of assignment 1:



(a) Average return vs episodes of sarsa , nstep sarsa



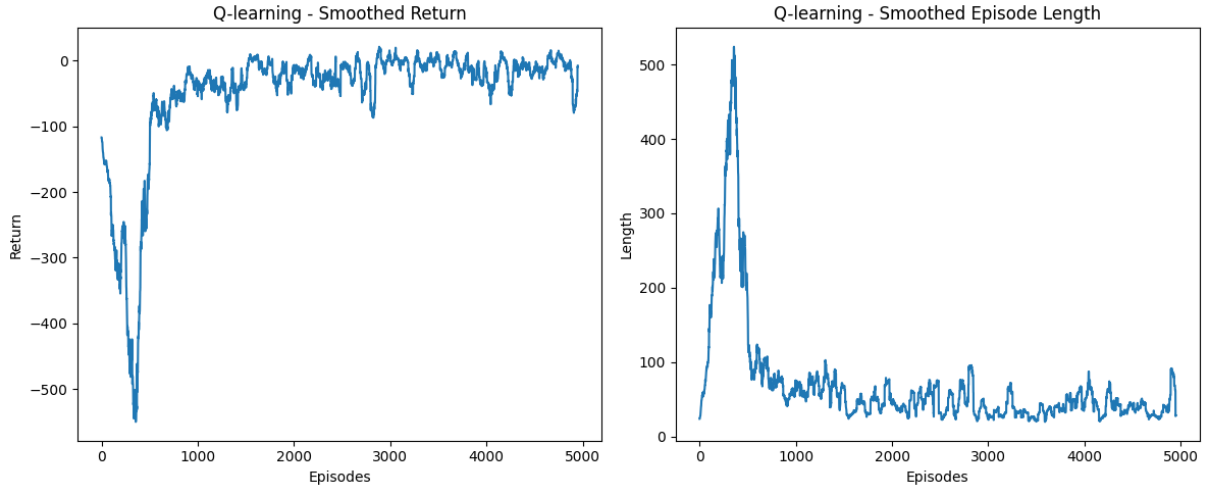
(b) Episode length vs episodes of sarsa , nstep sarsa

Figure 3: Comparison of SARSA and n-step SARSA as n increases

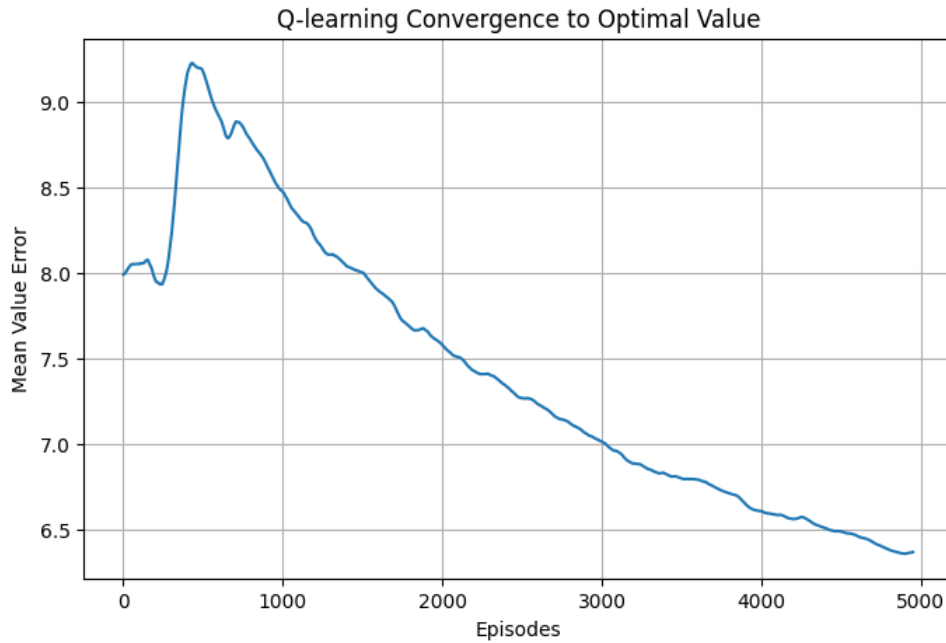
Method	Time (s)	Memory (KB)
SARSA	1.9406	264.33
n-step SARSA (n=3)	4.8482	300.33
n-step SARSA (n=5)	7.3661	456.48
n-step SARSA (n=10)	12.4259	562.78

Table 3: Comparison of Time and Memory Usage across Methods

Q2- Implementing Q1 of Assignment 1 in Qlearning
for question 1



(a) Average return and episode length vs episodes of q learning



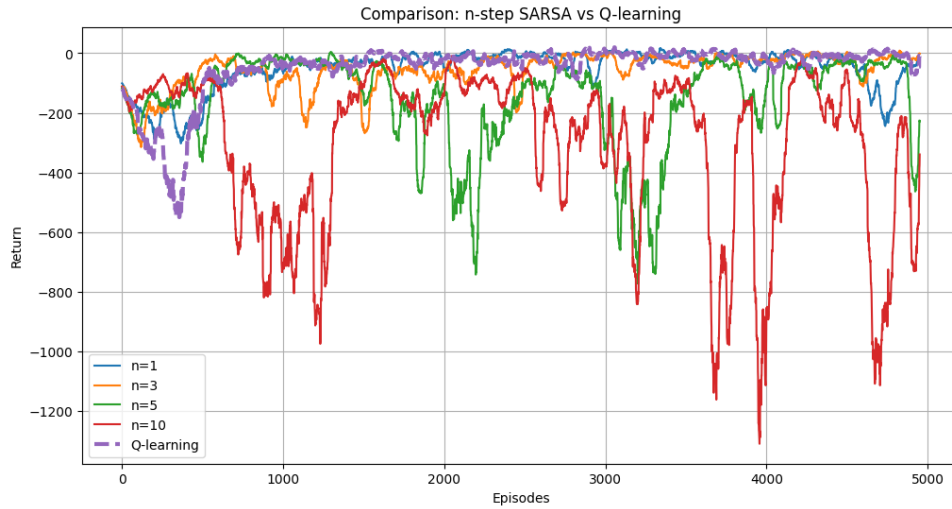
(b) Mean Value Error plot between Q learning and Policy iteration

Figure (a) shows the evolution of the average return and episode length over training episodes. Initially, the agent performs poorly, resulting in highly negative returns and large episode lengths due to random exploration. As training progresses, the return improves significantly and stabilizes near a higher value, indicating that the agent has learned a better policy. Simultaneously, the episode length decreases and stabilizes, suggesting that the agent reaches the goal more efficiently with fewer steps.

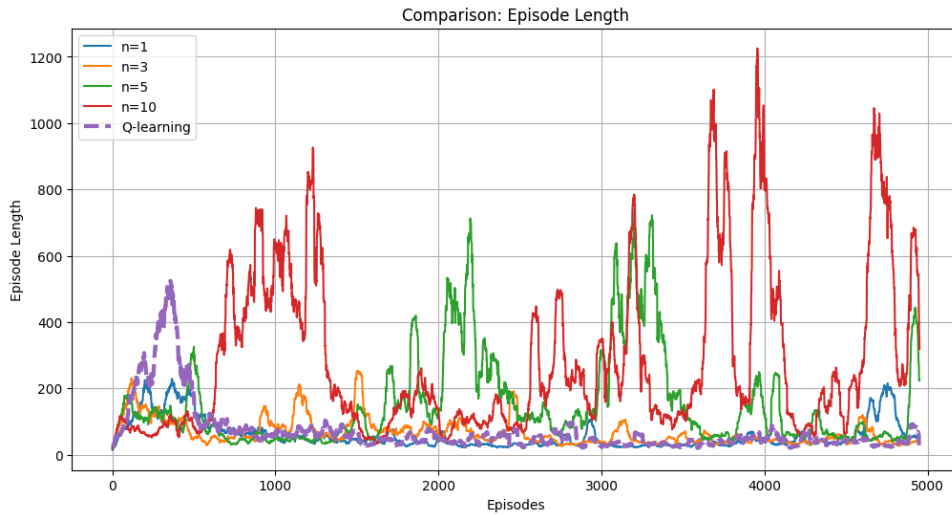
Figure (b) presents the convergence of Q-learning towards the optimal value function, measured via the mean value error with respect to the policy iteration solution. The error initially fluctuates due to exploration but gradually decreases over time, demonstrating convergence. The steady decline indicates that the learned Q-values are approaching the optimal values.

Overall, the results confirm that Q-learning successfully learns an optimal policy through iterative updates and converges over time, as evidenced by improved returns, reduced episode lengths, and decreasing value error.

Q3- Comparison of the performance of the algorithms given in assignment 1 and those using SARSA and Q-learning. [5]



(a) Average return vs episodes of sarsa , nstep sarsa, qlearning



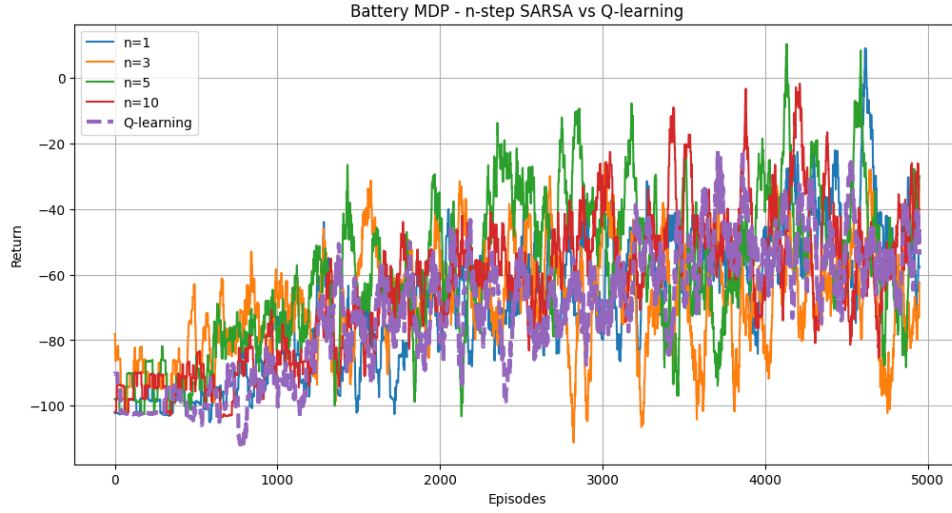
(b) Episode length vs episodes of sarsa , nstep sarsa, qlearning

Figure 5: Comparison of SARSA and n-step SARSA as n increases

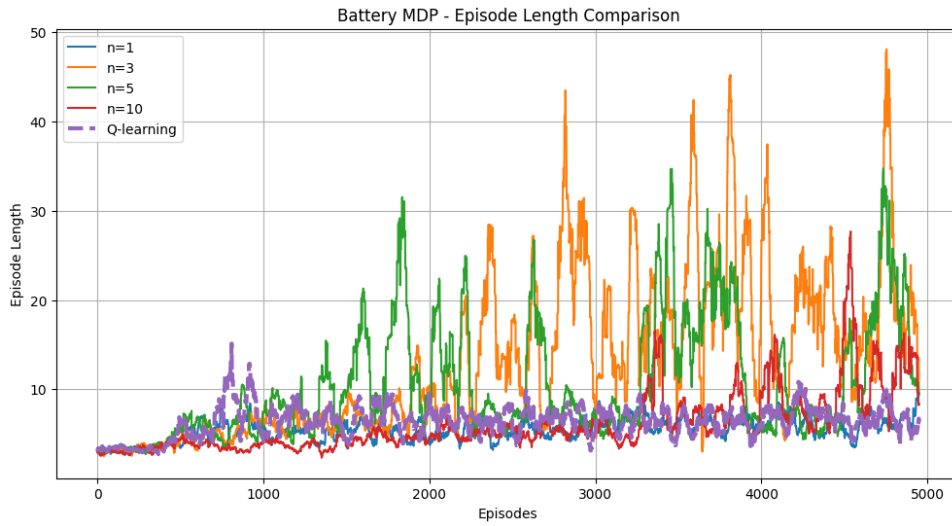
Method	Time (s)	Memory (KB)
Value Iteration	0.3508	56.68
Policy Iteration	1.2484	53.87
SARSA	9.0024	200.71
n-step SARSA (n=3)	14.4089	251.03
n-step SARSA (n=5)	29.5984	832.87
n-step SARSA (n=10)	55.5909	1045.01
Q-learning	15.0842	373.05

Table 4: Comparison of Time and Memory Usage across Methods

for question 2 (of assignment 1)



(a) Average return and episode length vs episodes of q learning

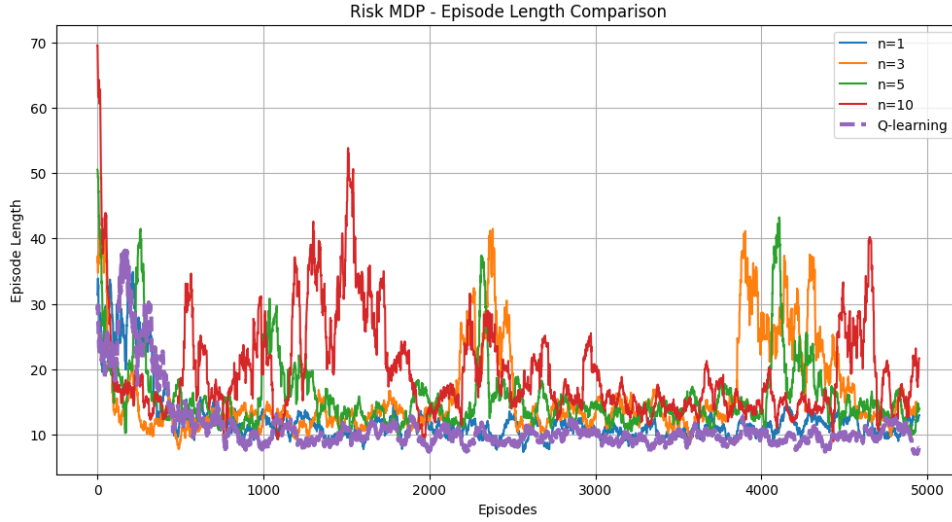


(b) Mean Value Error plot between Q learning and Policy iteration

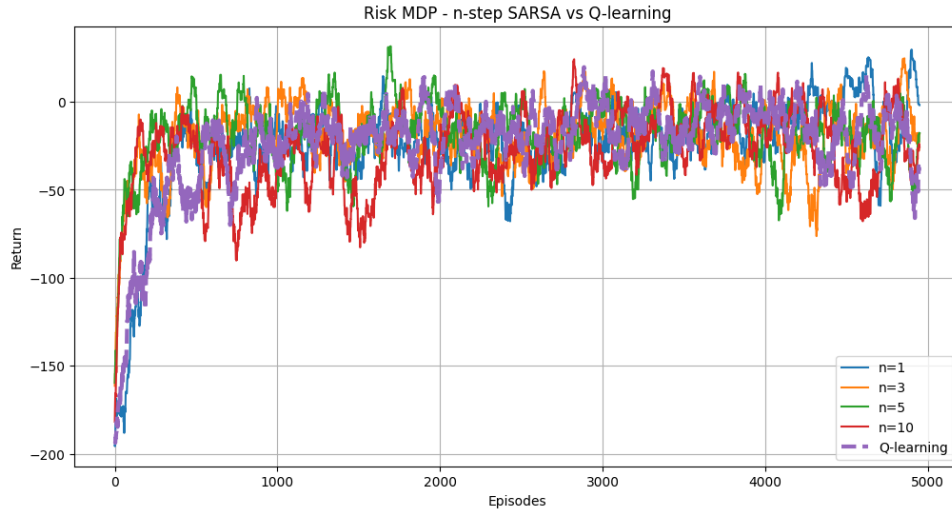
Method	Time (s)	Memory (KB)
SARSA	1.9406	264.33
n-step SARSA (n=3)	4.8482	300.33
n-step SARSA (n=5)	7.3661	456.48
n-step SARSA (n=10)	12.4259	562.78
Q-learning	16.0944	517.20

Table 5: Comparison of Time and Memory Usage across Methods

for question 3 (of assignment 1)



(a) Average return and episode leangth vs episodes of q learning



(b) Mean Value Error plot between Q learning and Policy iteration

Method	Time (s)	Memory (KB)
SARSA	3.7779	128.38
n-step SARSA (n=3)	5.3741	126.50
n-step SARSA (n=5)	5.7942	125.23
n-step SARSA (n=10)	7.0238	133.23
Q-learning	5.4969	300.80

Table 6: Comparison of Time and Memory Usage across Methods

Comparison with Value and Policy Iteration

- **Efficiency:** Value Iteration and Policy Iteration are significantly faster (sub-second runtime) and require much less memory compared to SARSA and Q-learning, as they directly use the known model of the environment.

- **Model-based vs Model-free:** Value and Policy Iteration are model-based methods, allowing them to compute optimal policies deterministically. In contrast, SARSA and Q-learning are model-free and rely on sampling, leading to higher computational time and memory usage.
- **Convergence Behavior:** From the plots, Q-learning (purple line) converges more smoothly and faster compared to SARSA and n-step SARSA. SARSA and especially higher n-step variants show high variance and instability in both return and episode length.
- **Stability:** SARSA (on-policy) is more stable than n-step SARSA with larger n , but still slower compared to Q-learning. Large n values introduce high variance, causing spikes in episode length and large negative returns.
- **Performance Trade-off:** Q-learning achieves better convergence and stability than SARSA while maintaining moderate computational cost. However, both are less efficient than Value and Policy Iteration due to the need for exploration.
- **Scalability:** Although Value and Policy Iteration are efficient in small environments, they do not scale well to large state spaces. In contrast, SARSA and Q-learning are more scalable but come at the cost of slower convergence and higher resource usage.

Q4- Given a grid of 1000x1000 with randomly placed obstacles occupying 10% of the grid. In this environment, start and goal locations are randomly placed. The objective is to find an optimal policy using any of the methods studied in the class so far.

Choice of Algorithm: Q-Learning

Q-learning, a model-free reinforcement learning algorithm, is used to learn the optimal policy.

Justification:

- **No Model Requirement:** The transition dynamics of the environment are not explicitly known. Q-learning does not require a model, unlike Value Iteration (VI) and Policy Iteration (PI), which need full knowledge of state transitions and rewards.
- **Large State Space:** The grid contains 10^6 states. VI and PI become computationally expensive due to full state sweeps, whereas Q-learning updates only visited states, making it more scalable.
- **Online Learning:** Q-learning learns directly through interaction with the environment, making it suitable for this setting where the agent explores the grid incrementally.
- **Comparison with Other Methods:**
 - **Monte Carlo (MC):** Requires complete episodes and is inefficient in long-horizon problems.
 - **SARSA:** Being on-policy, it learns more conservatively and converges slower compared to Q-learning.

- **n-step SARSA:** Improves learning speed but still remains on-policy and less sample-efficient than Q-learning.

Methodology:

The agent state is defined as (r, c) and actions correspond to four directions (up, down, left, right). The Q-values are updated using:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right]$$

An ϵ -greedy policy is used for exploration with decay over episodes.

Reward Design:

- +100 for reaching the goal
- -5 for hitting obstacles
- +1 for moving closer to the goal
- -1 for moving away
- -0.1 for no progress

Results:

- The agent successfully learns a path from start to goal.
- The extracted path is efficient and avoids obstacles.
- Training rewards increase over time and stabilize.
- Success rate converges to approximately 90% – 95%.

Q-learning effectively learns a near-optimal navigation policy in a large grid environment without requiring prior knowledge of the environment. Its ability to perform online, model-free learning makes it more suitable than VI, PI, MC, and SARSA-based methods for this problem.

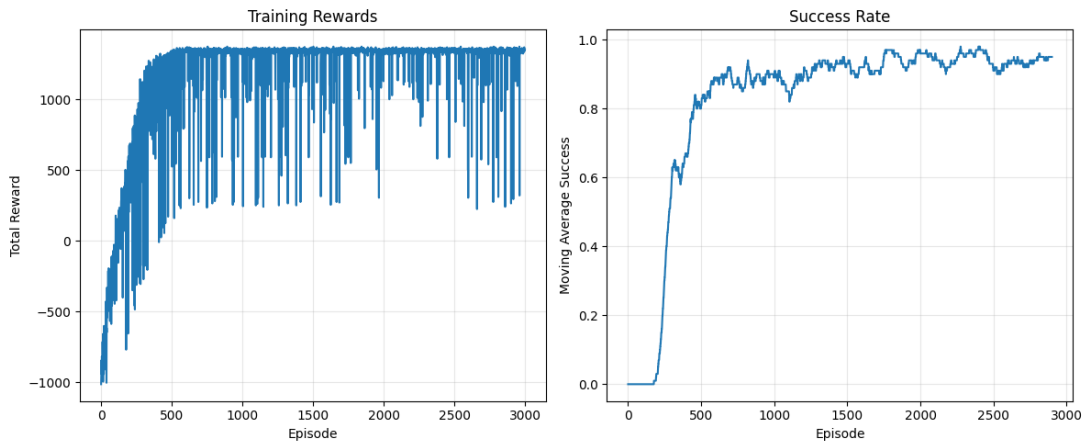


Figure 8: Raw Rewards and Success Rates over Episodes

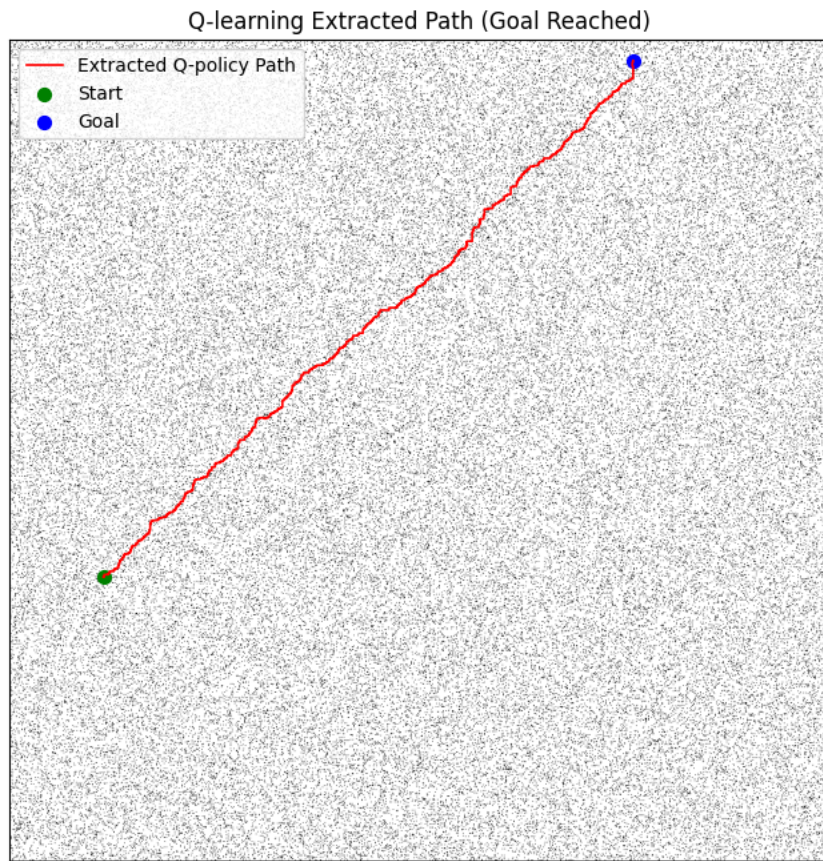


Figure 9: Demonstration

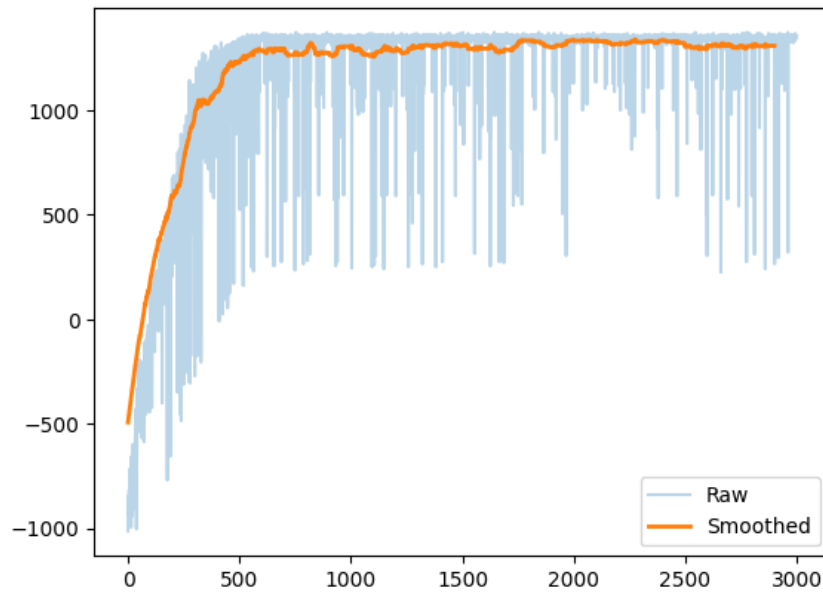


Figure 10: Smoothed average reward w.r.t Episodes